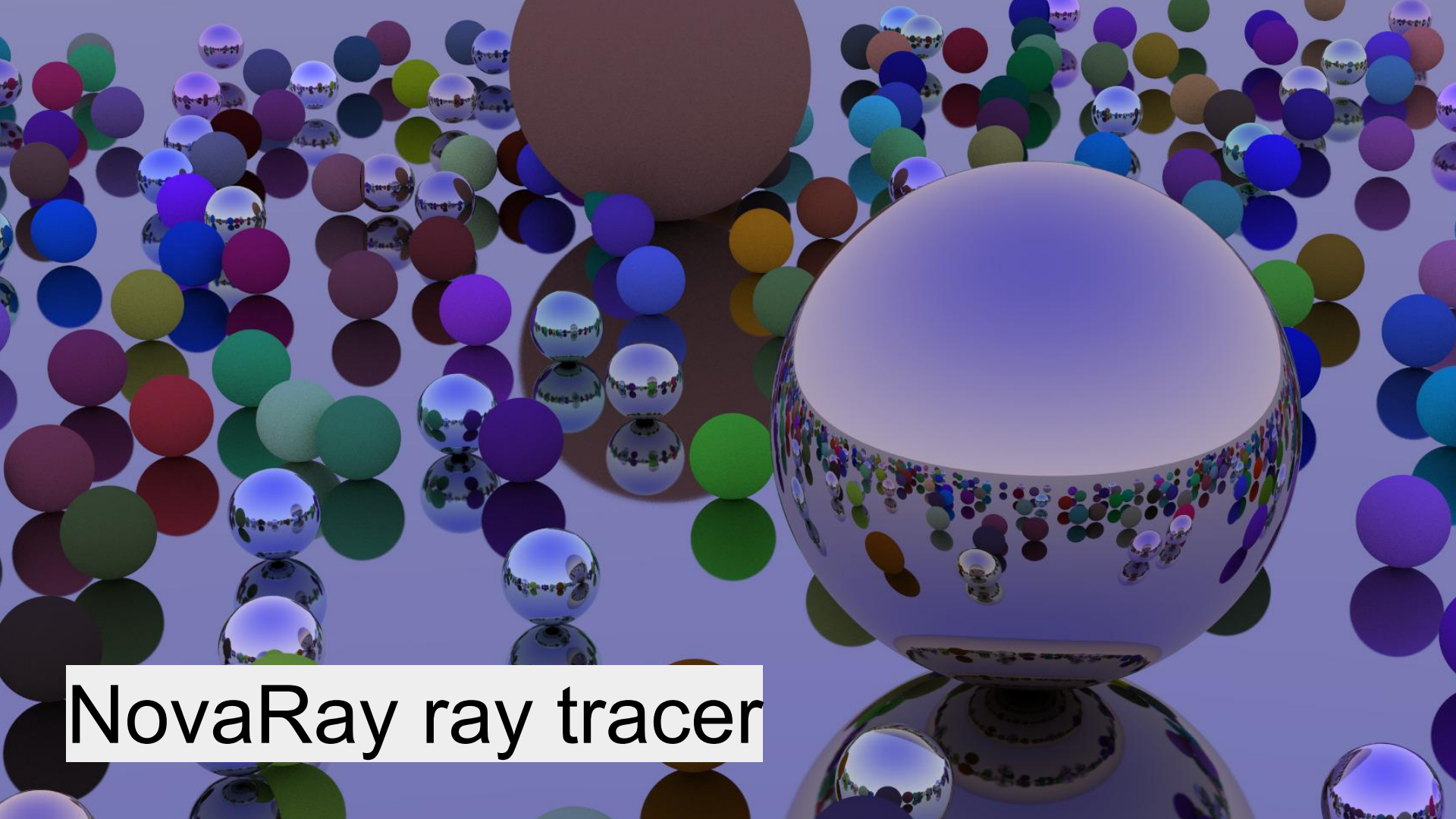




NovaRay ray tracer

Why ray tracer?

- Since SysPro doesn't have some computer graphics course, we became curious about exploring and solving new interesting problems.

Why ray tracer?

- Since SysPro doesn't have some computer graphics course, we became curious about exploring and solving new interesting problems.
- It seems like such field requires some algorithmic and math background (all of us are actually mathematicians)

Why ray tracer?

- Since SysPro doesn't have some computer graphics course, we became curious about exploring and solving new interesting problems.
- It seems like such field requires some algorithmic and math background (all of us are actually mathematicians (at least for an upcoming exam period)).

Why ray tracer?

- Since SysPro doesn't have some computer graphics course, we became curious about exploring and solving new interesting problems.
- It seems like such fields require some algorithmic and math background (we are all actually mathematicians (at least for an upcoming exam period)).
- It is kinda satisfying to see the result of your work almost on the fly.

And after all

Why ray tracer?

- Since SysPro doesn't have some computer graphics course, we became curious about exploring and solving new interesting problems.
- It seems like such fields require some algorithmic and math background (we are all actually mathematicians (at least for an upcoming exam period)).
- It is kinda satisfying to see the result of your work almost on the fly.

And after all



(tbh no, it's not)



Our goals

We had to implement ray tracer with following functionality:

- primitives: planes and spheres (what to render)
- materials: matte and metals (how to make primitives look more realistic and interesting)
- anti-aliasing, motion blur and light (how to make materials more natural)
- arbitrary camera positioning (how to make work with scenes more flexible)
- boundary volume hierarchy (how to speed up rendering with better ray-object intersection algorithm)
- tiling based rendering



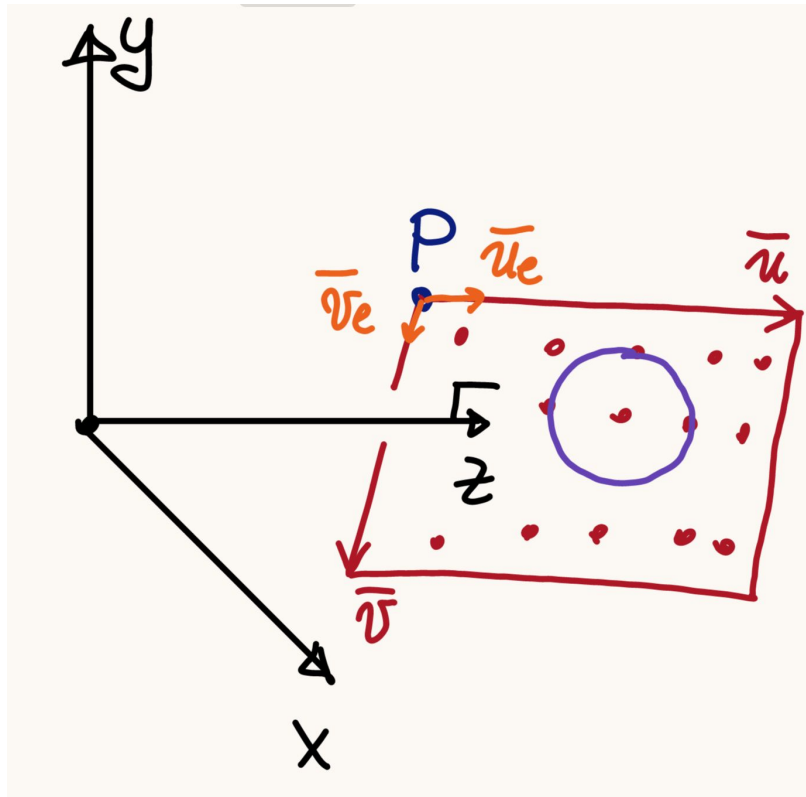
Our goals

We had to implement ray tracer with following functionality:

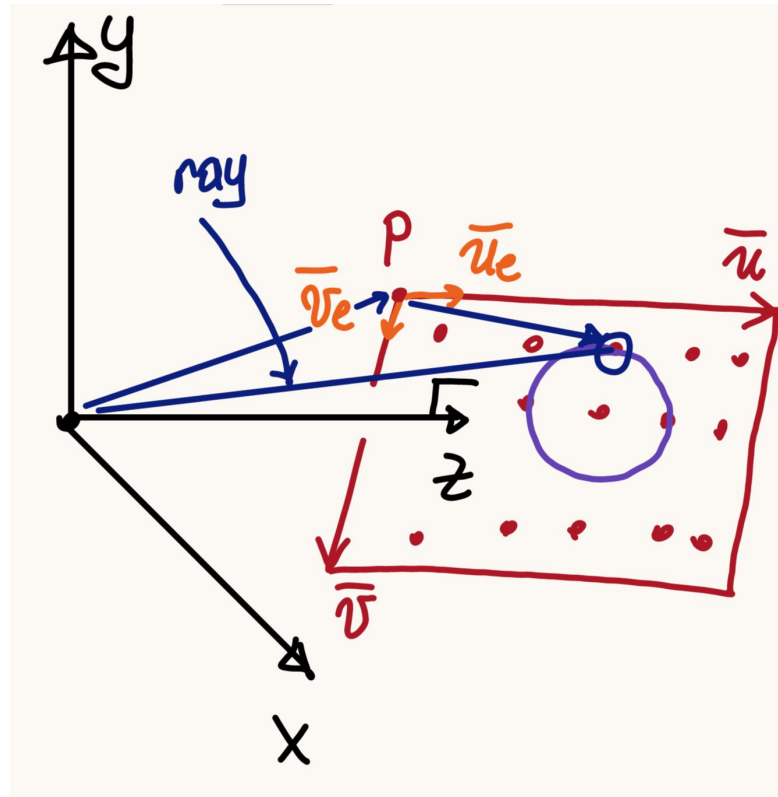
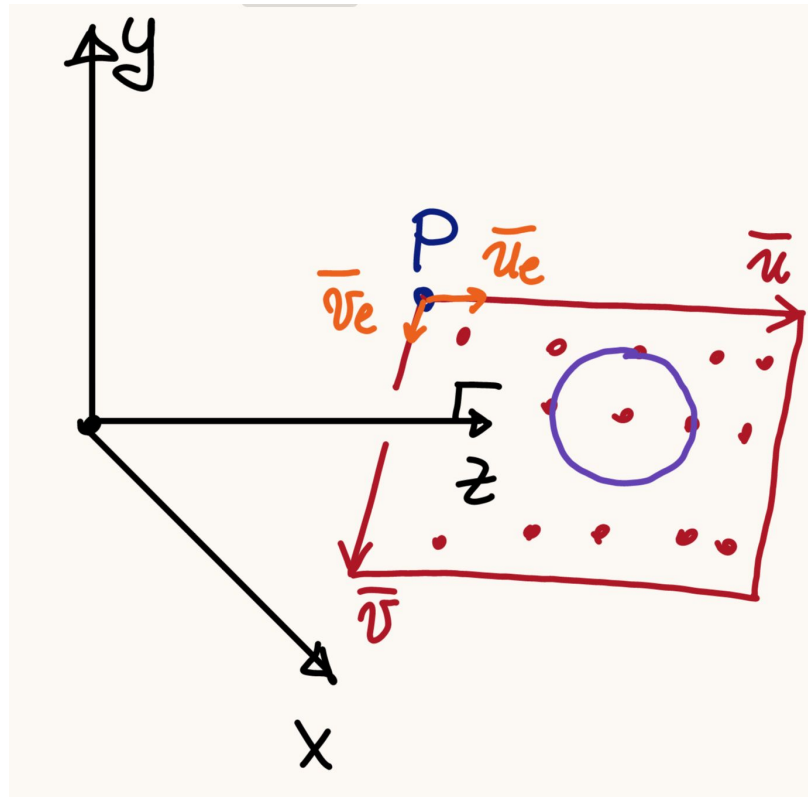
- primitives: planes and spheres (what to render)
- materials: matte and metals (how to make primitives look more realistic and interesting)
- anti-aliasing, motion blur and light (how to make materials more natural)
- arbitrary camera positioning (how to make work with scenes more flexible)
- boundary volume hierarchy (how to speed up rendering with better ray-object intersection algorithm)
- tiling based rendering

Let's go down to the business

Little warmup



Little warmup



Intersection with objects

The algorithm of finding an intersection with some 3D object consists of the following steps (in majority of cases):

- to find out an equation (or system of equations) of an object (in case of manifolds it's kinda easy).
- substitute the ray equation (which is simply parameterized equation of line $P(t) = Q + t * d$, where Q is the origin, d - direction vector, t - parameter) into object equation.
- solve with respect to parameter (t in our case).

Intersection with objects

For example sphere with center in point $C = (C1, C2, C3)$ of radius r has such equation:

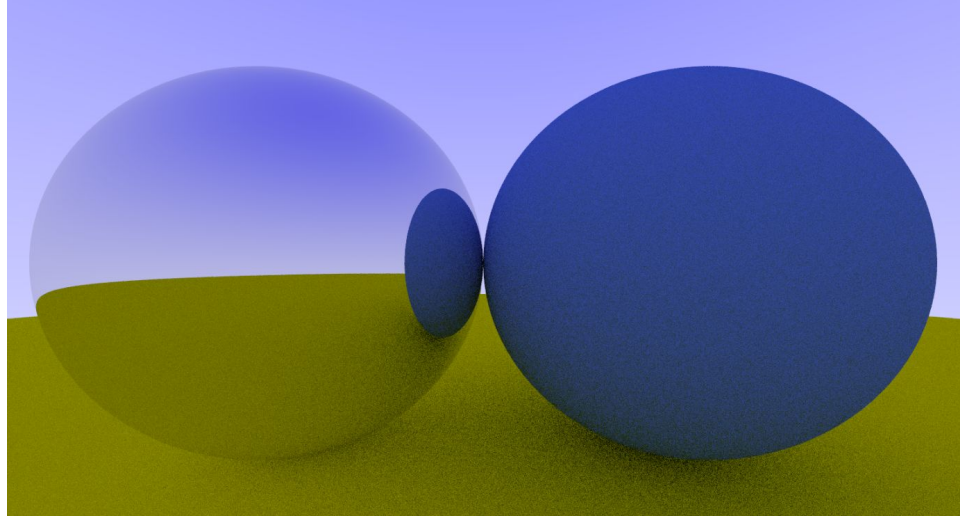
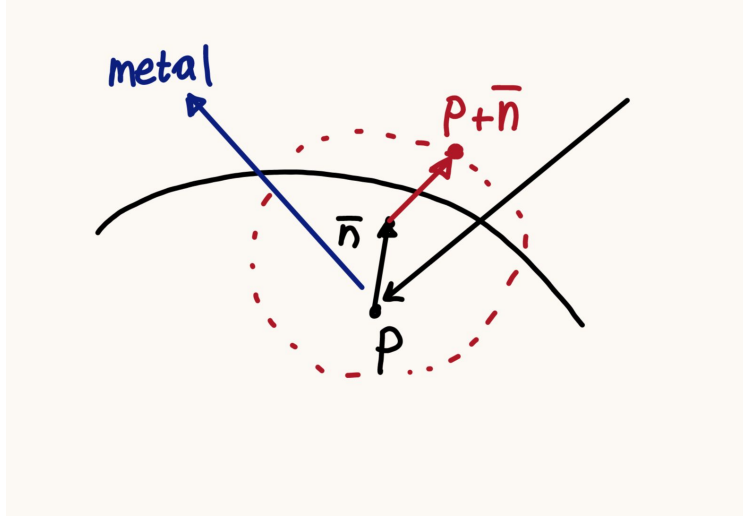
$$(C1 - x)^2 + (C2 - y)^2 + (C3 - z)^2 - r^2 = 0.$$

$$(\mathbf{C} - \mathbf{P}) \cdot (\mathbf{C} - \mathbf{P}) - r^2 = 0.$$

$$t^2 \mathbf{d} \cdot \mathbf{d} - 2t \mathbf{d} \cdot (\mathbf{C} - \mathbf{Q}) + (\mathbf{C} - \mathbf{Q}) \cdot (\mathbf{C} - \mathbf{Q}) - r^2 = 0.$$

Obvious + easy to notice + proof left for readers as simple exercise. ■

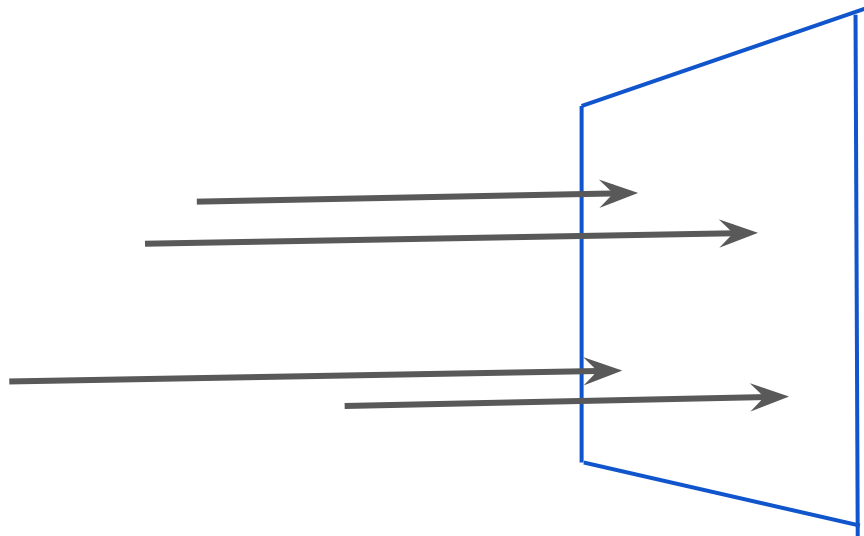
Materials: metals and matte (diffuse materials)



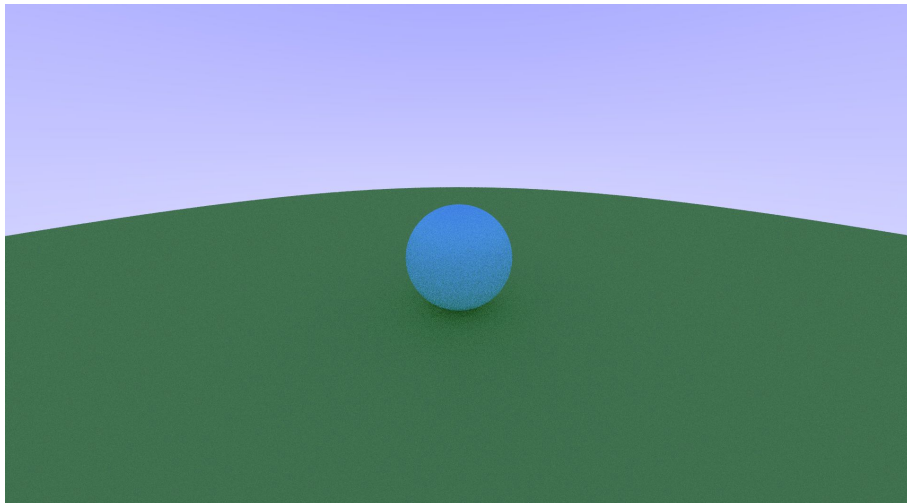
Anti-aliasing



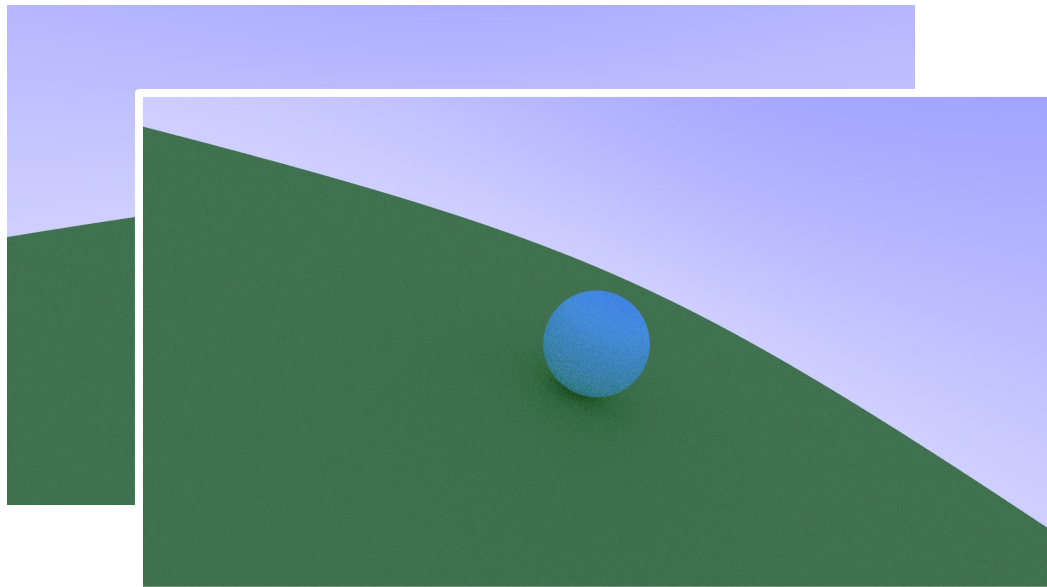
Anti-aliasing



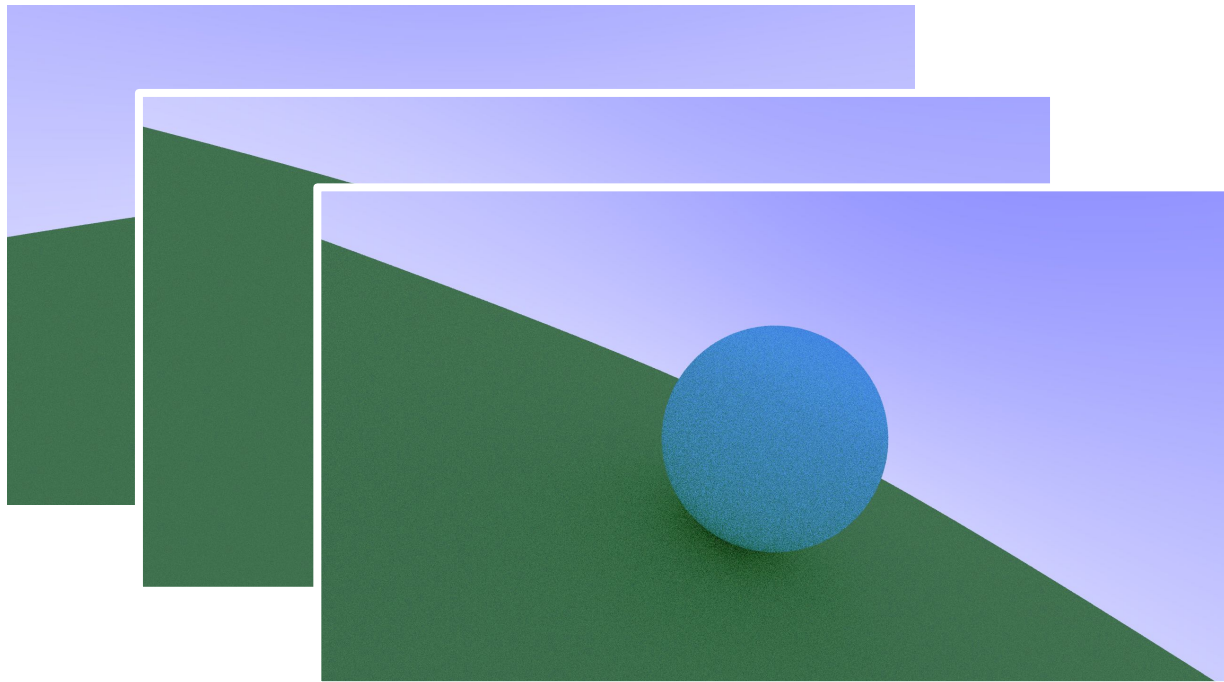
Camera



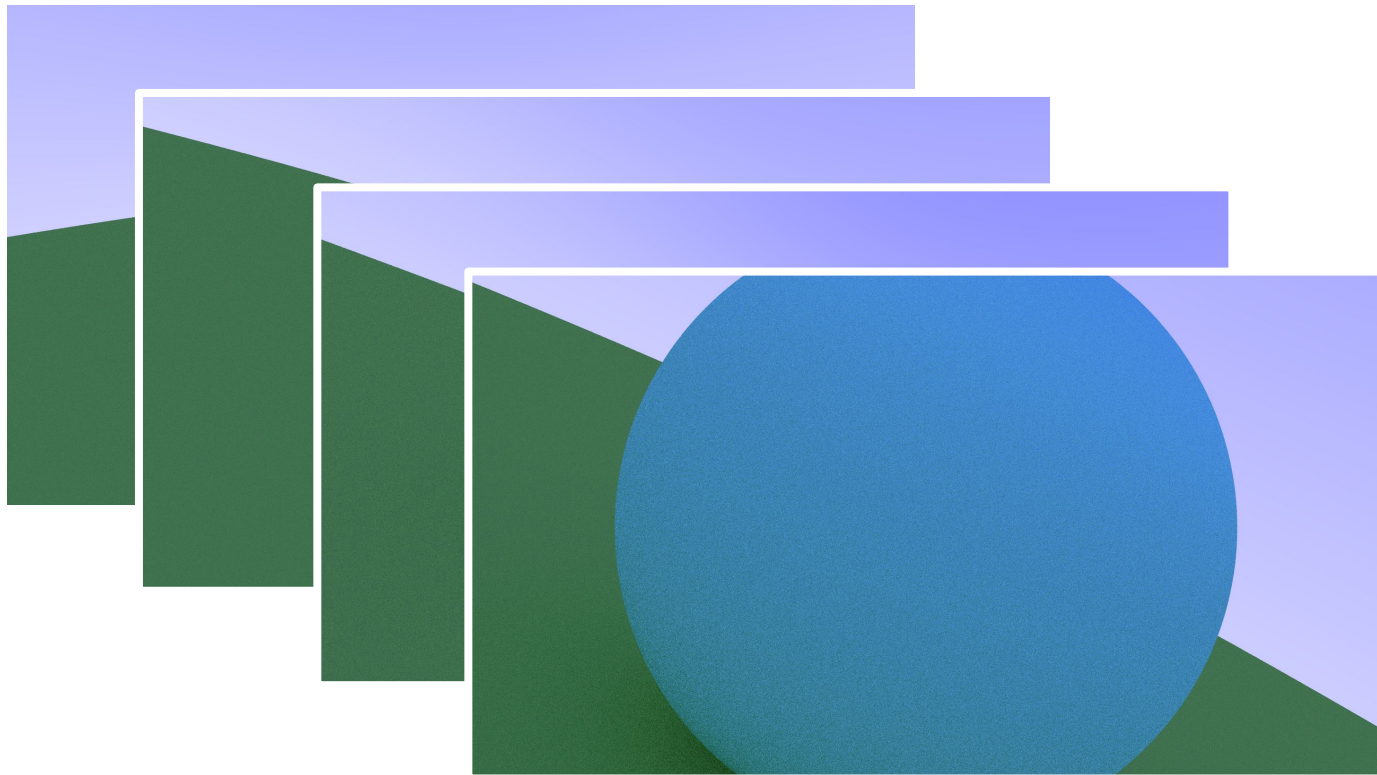
Camera



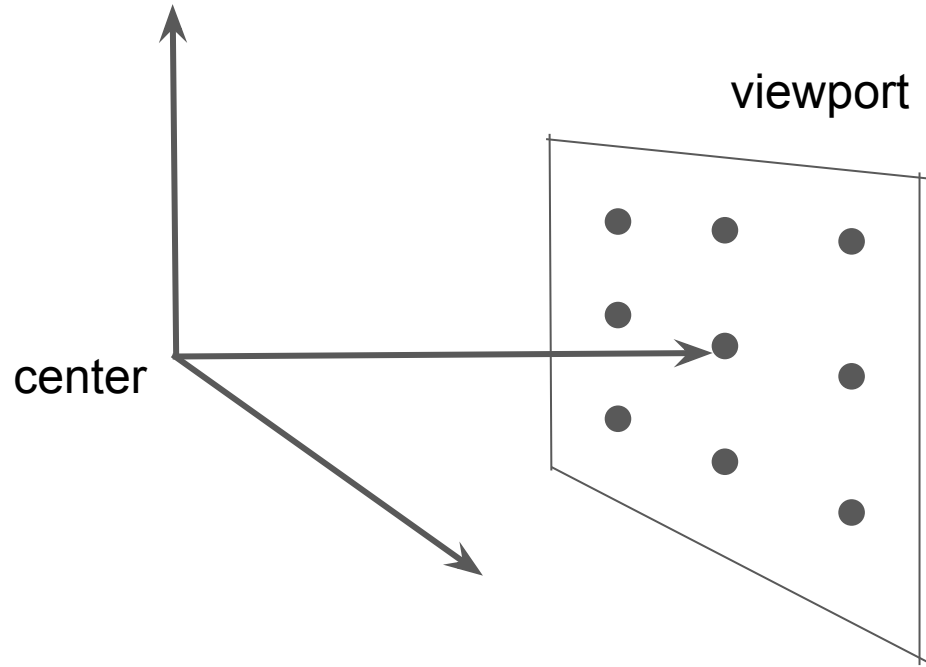
Camera



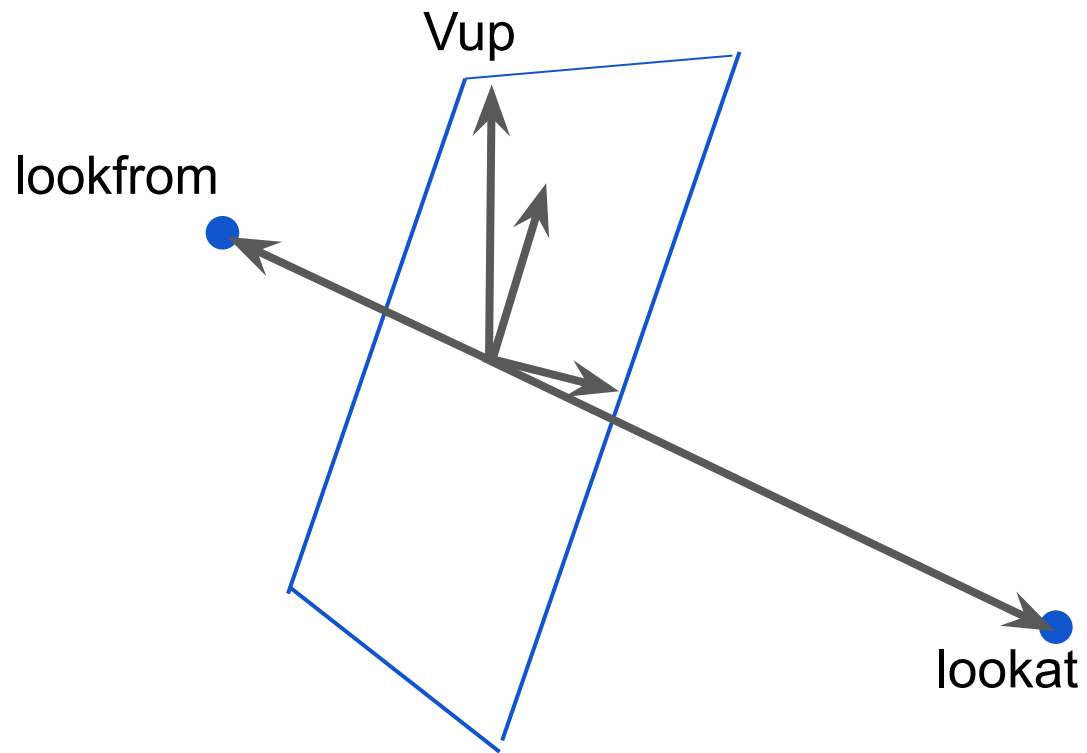
Camera



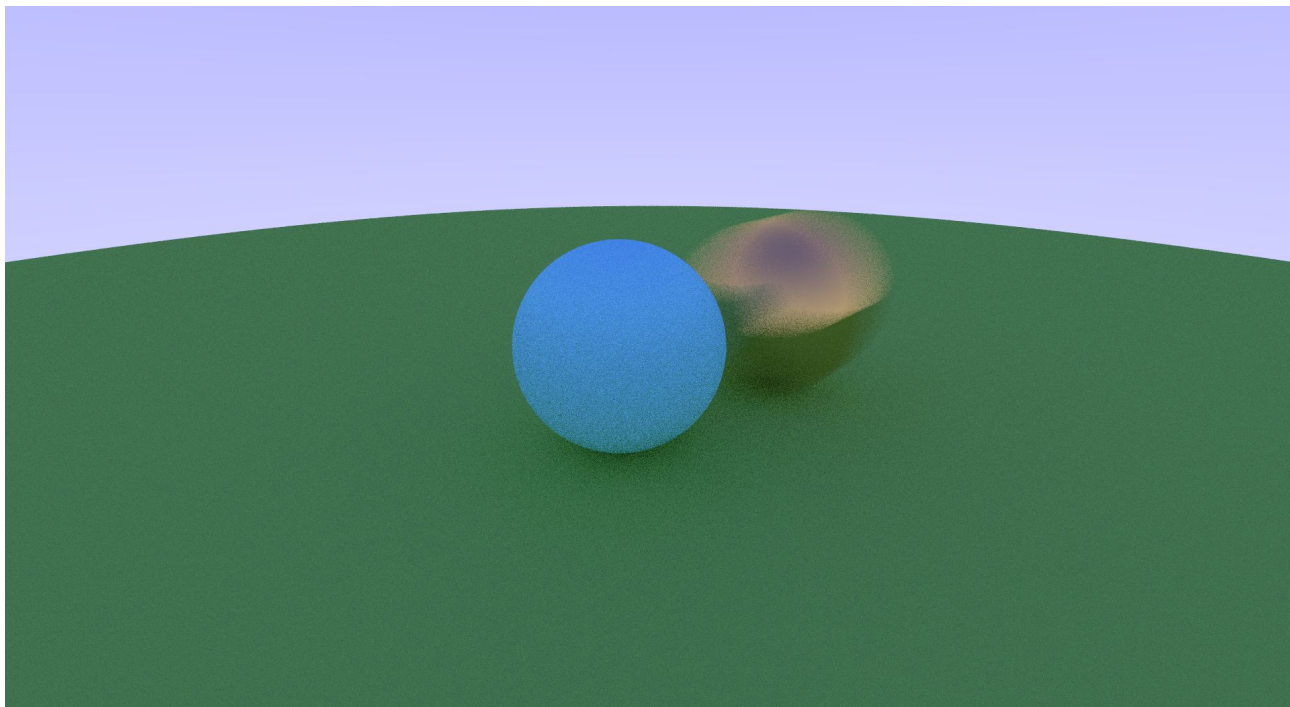
Camera previously



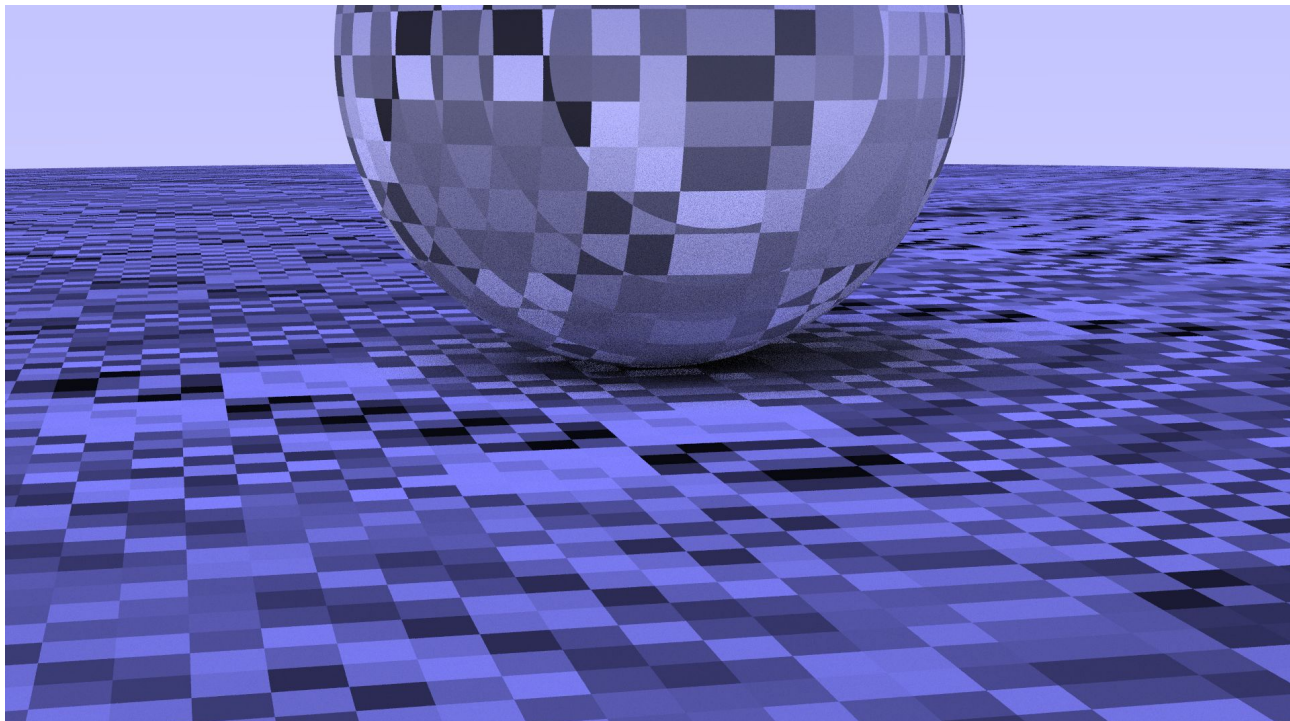
Camera now



Motion blur



Perlin noise v1

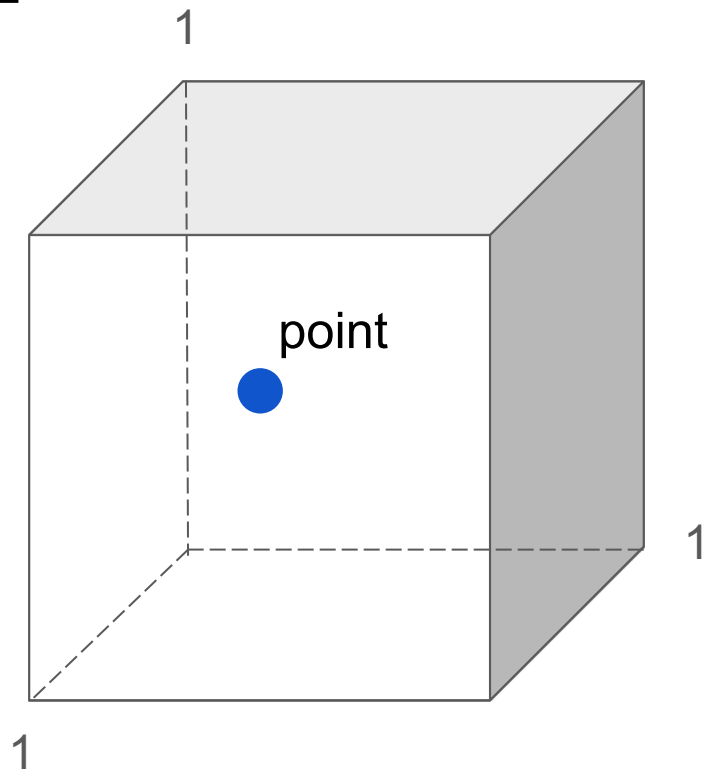


Perlin noise v1

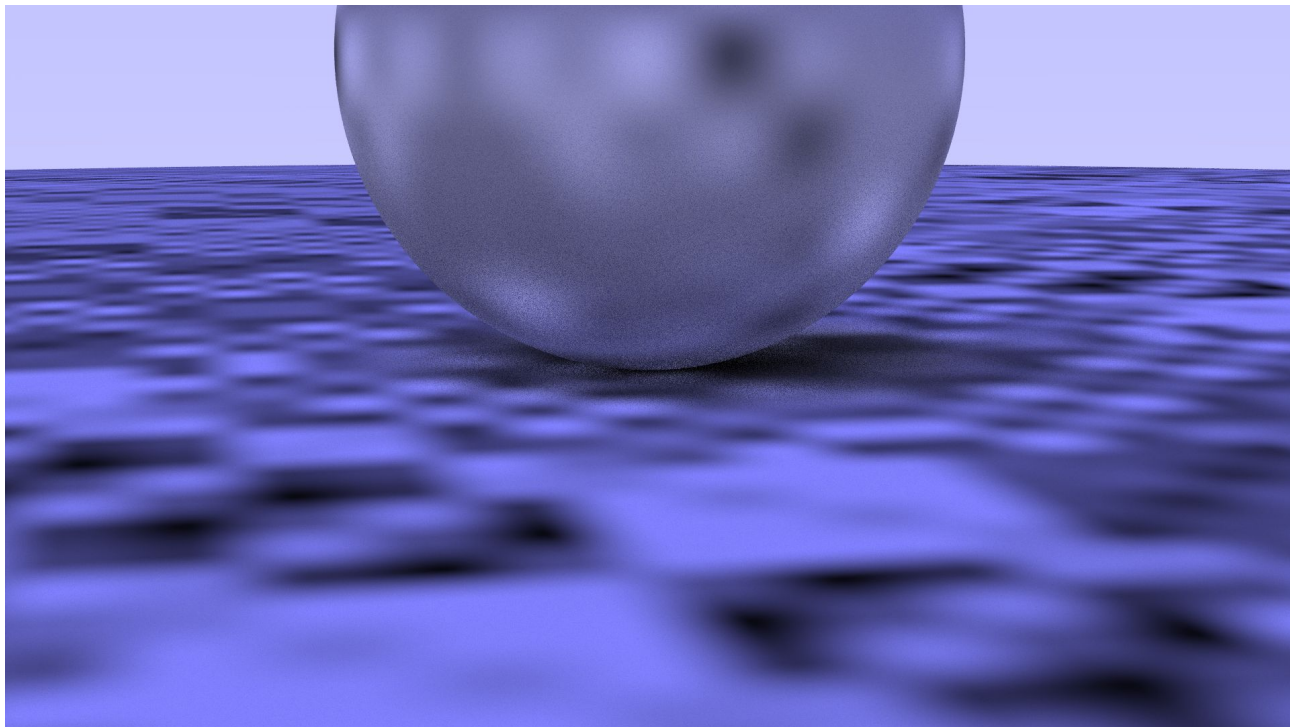
How it works?

- 1) creating an array of 256 elements with random numbers
- 2) generating three arrays with permutations of numbers from 0 to 255
- 3) selecting an element from the first array using as an index a combination of values from the other three arrays with the indexes as the coordinates of the point

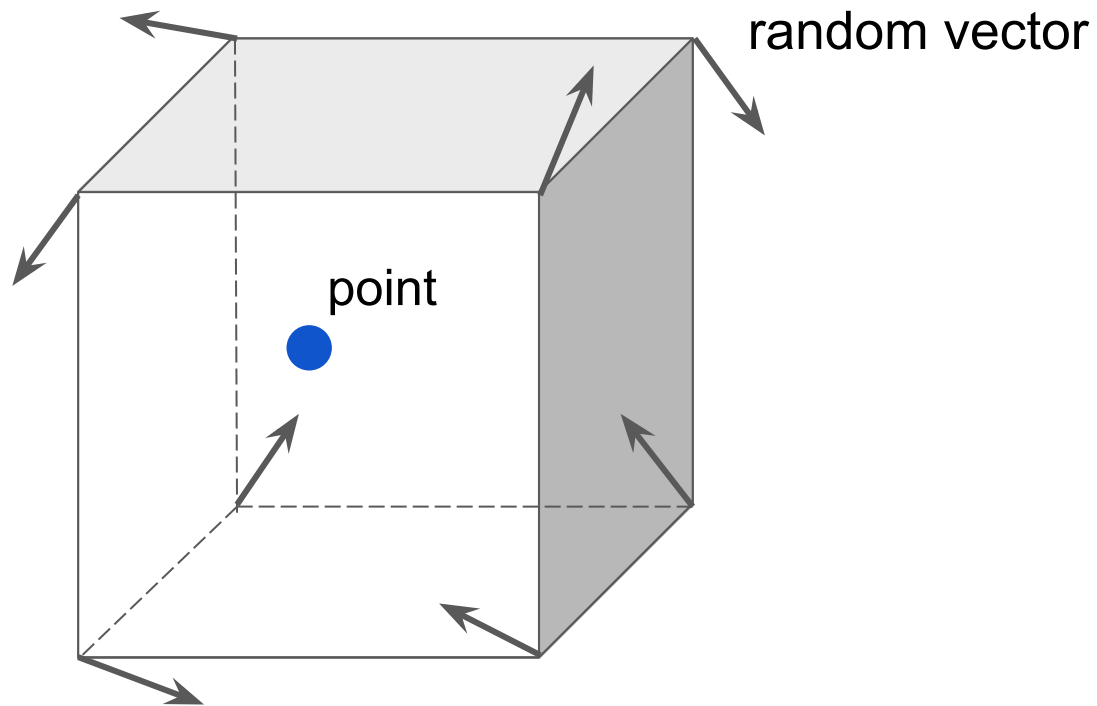
Perlin noise v2



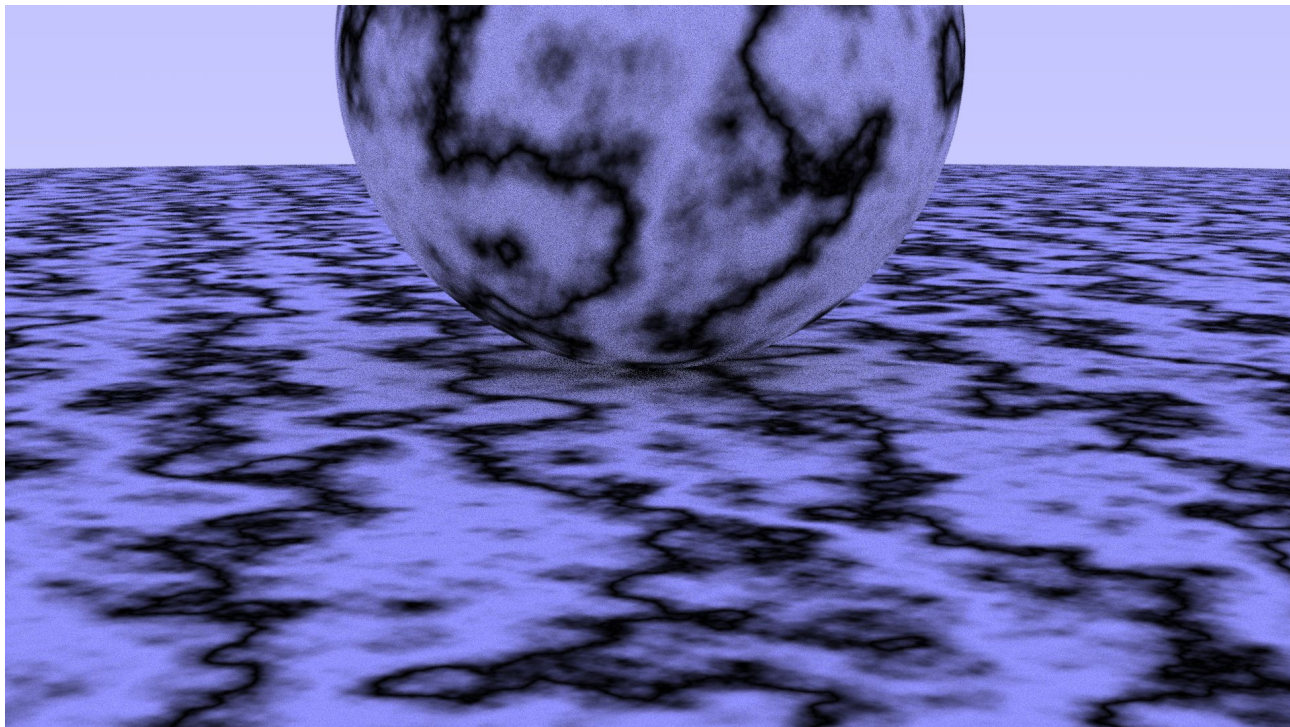
Perlin noise v2



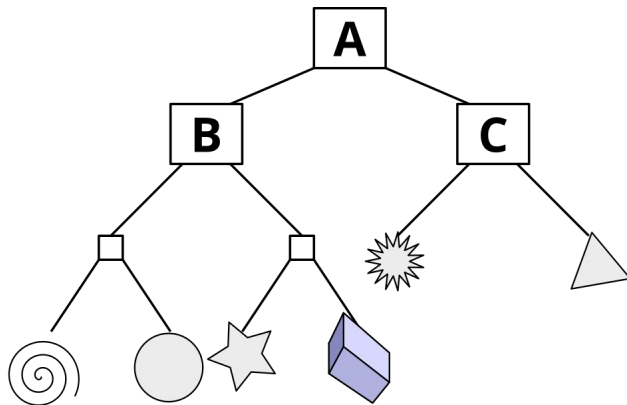
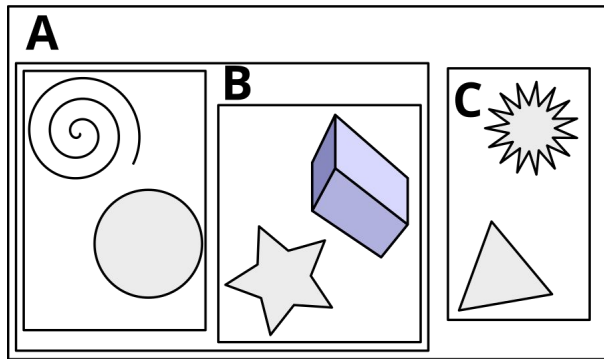
Perlin noise v3



Perlin noise v3 ✨



Bounding volume hierarchy: idea



hit:

if ray does not intersect hitbox then return

try to find in left subtree

try to find in right one

Bounding volume hierarchy: how to build it?

Let's use divide and conquer approach!

We need to *somehow* group volumes, sort it and put half in each subtree.

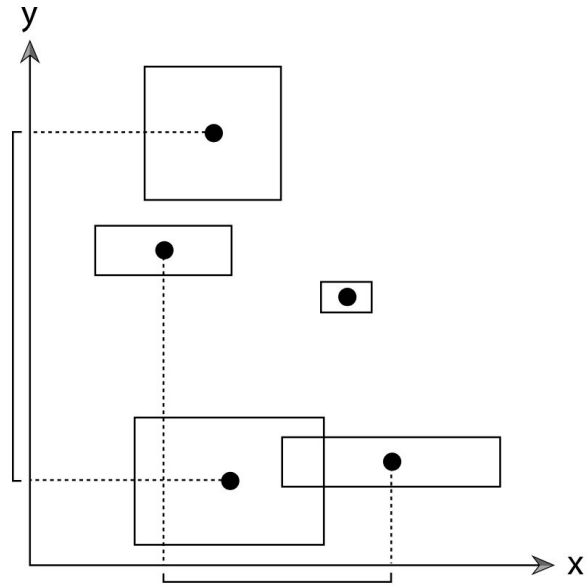
So, there are several approaches:

- “cut” screen into tiles
- use “random axis” approach
- axis-aligned minimum bounding box (we will use this one)
- some more complicated but more adaptive approaches like covariance matrices, etc.

Bounding volume hierarchy: splitting volumes

The algorithm of building BVH tree (recursive part is very similar to merge sort) :

1. create “parent node hitbox”, which fully contains children hitboxes
2. sort an array along the longest axis
3. call build from subtrees



Bounding volume hierarchy: benchmarking

Objects / ms	Linear intersection algo	BVH intersection algo
100	12	9
1000	100	31
10000	829	225
20000	1571	457

Tile based rendering: how to engage more threads

Tile based rendering: how to engage more threads and gain more power

The idea is quite simple: just divide screen into *tiles* and render them in parallel



Tile based rendering: how to engage more threads and gain more power

The idea is quite simple: just divide screen into *tiles* and render them in parallel

I guess, the only challenge i've encountered is about broken rendering pipeline



Tile based rendering: how to engage more threads and gain more power

The idea is quite simple: just divide screen into *tiles* and render them in parallel

I guess, the only challenge i've encountered is about broken rendering pipeline

Thank God that in such setup we have no data races on BVH tree

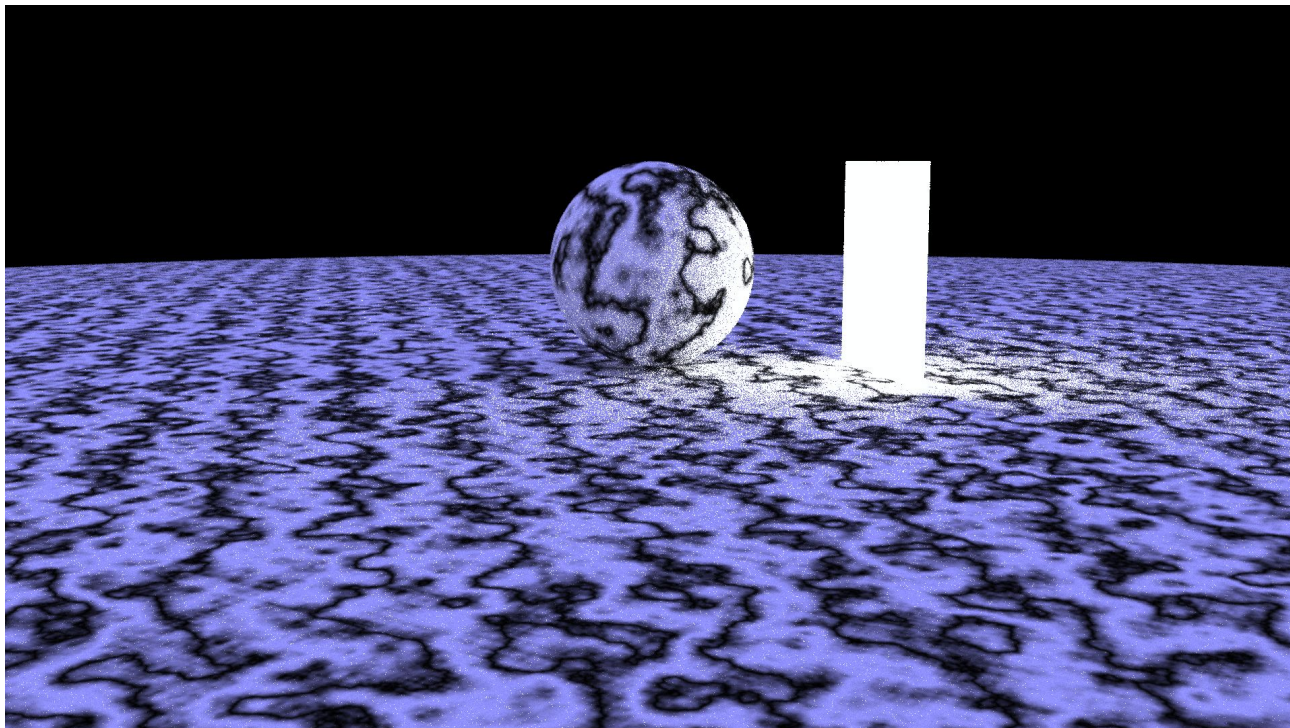


Tile based rendering: benchmarks

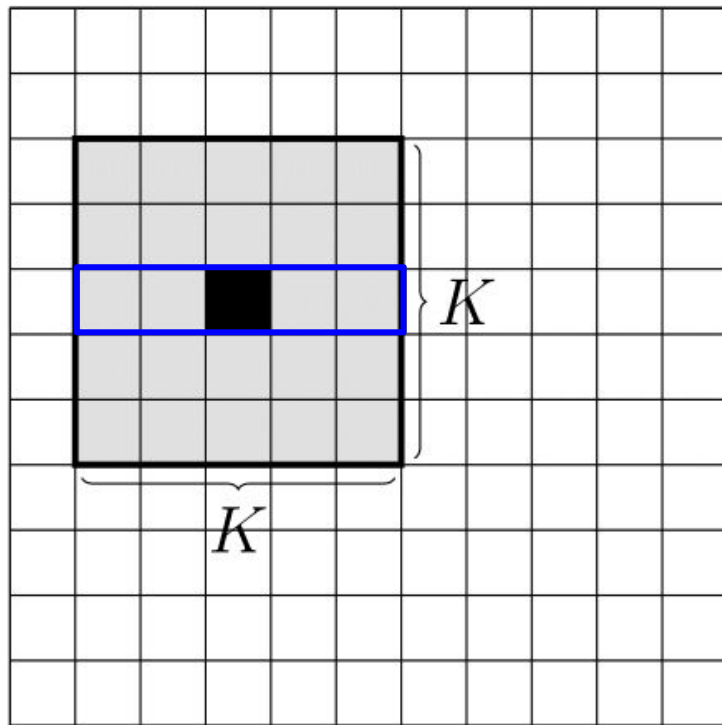
Setup for image: 1920x1080, 20 samples per pixel, bounces limit – 20.

Resources/ Scene	Metals (5 objects)	Levitating (25 objects)	Bulky (50 objects)	Bu-bulky (1000 objects)
single thread	130	90	215	Until Bloodborne on PC (i guess never)
8 threads, 16 tiles	30	24	70	130
8 threads, 32 tiles	51	37	84	159

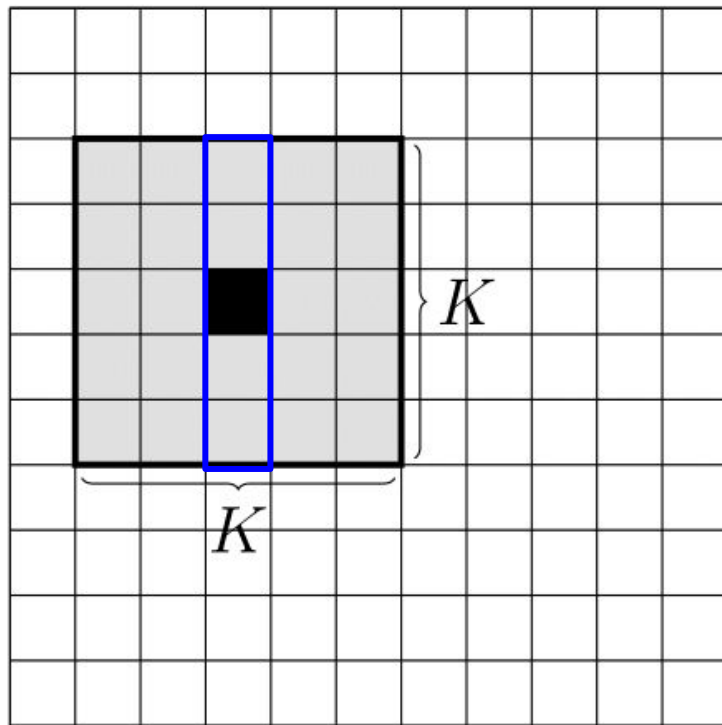
Light



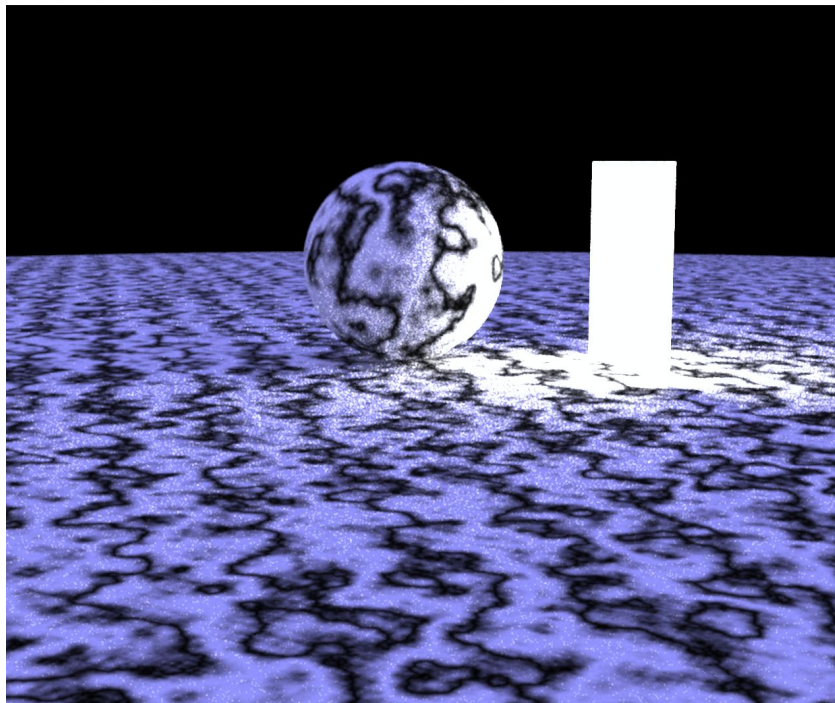
Denoiser



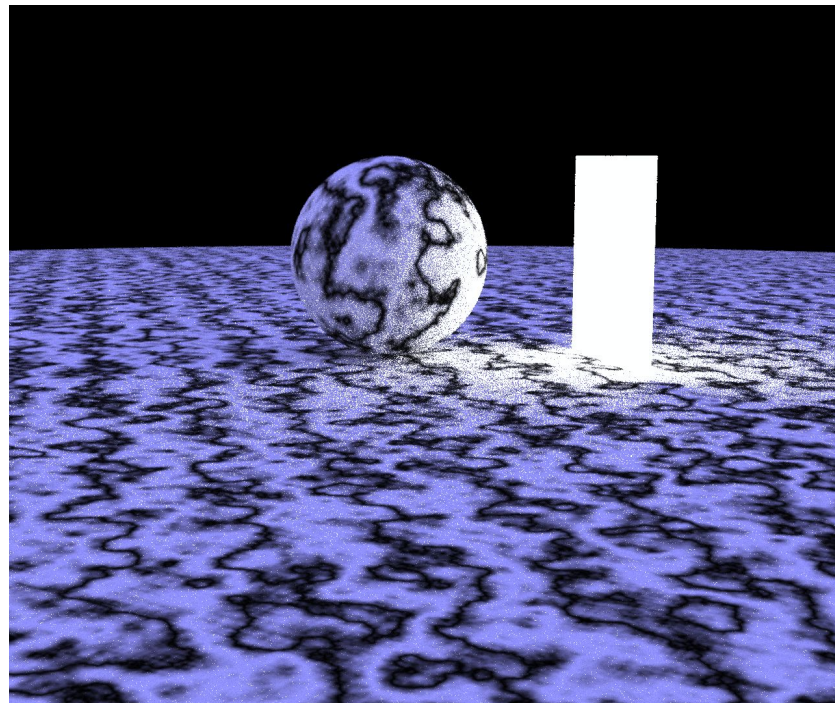
Denoiser



Denoiser



with



without

Conclusion

- interesting math (usually hard to understand)
- many things that can be implemented:some intersection algorithms that use advanced integration techniques(Monte Carlo integration, for example)
- to provide more primitives(unusual manifolds) we need more advanced algorithms for creating bounding volumes(oriented bounding box)